The following example uses a throw to terminate interaction with the user if ! is typed in response to any prompt:

tut_exceptions/catchthrow.rb

```
def prompt_and_get(prompt)
  print prompt
  res = readline.chomp
  throw :quit_requested if res == "!"
  res
end

catch :quit_requested do
  name = prompt_and_get("Name: ")
  age = prompt_and_get("Age:  ")
  sex = prompt_and_get("Sex:  ")
  # ..
  # process information
end
```

As this example illustrates, the throw does not have to appear within the static scope of the catch.

What's Next

In this chapter, we looked at how to make our Ruby code more error-proof by catching and raising exceptions, and you saw how to create your own exception classes that might have their own data. Next up, we're going to talk about a leading cause of exceptions in code: managing input and output.